

Slew Guard: Real-Time voltage protection for GPU workloads

Members: Nikhil Reddy Mummadi, Jakub Mroczek

Abstract

Slew Guard is a closed-loop software controller that protects a GPU's power-delivery network (PDN) from load-voltage droops caused by rapid workload current changes (di/dt). The system runs an end-to-end ResNet-18 workload on a live GPU, models the PDN as an RLC circuit with an envelope filter driven by NVML telemetry, and throttles the kernel's batch size in real time so the modelled load voltage (V_{load}) never falls below a user specified minimum voltage (V_{min}). A hysteresis controller with a tunable deadband issues shrink or grow decisions every 50 μs based on a single operational threshold V_{shrink} derived from $V_{safe} = V_{min} + 30 \text{ mV}$. All PDN parameters for DC power level, noise amplitude, and derived voltage thresholds are calibrated automatically from live NVML measurements at startup.

Background and Motivation

Modern deep learning workloads place unprecedented peak-power demands on GPU power-delivery networks (PDNs). As a neural network advances layer by layer the GPU alternates between periods of high and low compute activity. Each transition generates a rapid current step (di/dt) that the PDN must absorb without letting the load voltage V_{load} fall below the minimum safe operating voltage V_{min} . When $V_{load} < V_{min}$, timing violations occur and the GPU risks incorrect computation or a hard crash.

The PDN behaves as a lumped second-order RLC circuit. Its load voltage is $V_{load} = V_{nom} - I \cdot R - \Delta I \cdot Z_{LC} \cdot \text{overshoot_factor}$ where R is the series resistance, L is the package/socket inductance, C is the bulk decoupling capacitance, $Z_{LC} = \sqrt{L/C}$ is the characteristic impedance ($\approx 3.87 \text{ m}\Omega$ for $L = 15 \text{ nH}$, $C = 1 \text{ mF}$), and ΔI is the current step at the start of a new compute phase. For a V100-class GPU drawing 200 W at 0.9 V, ΔI can exceed 200 A, producing peak droops of 50–100 mV within the resonant period $T_{res} = 2\pi\sqrt{LC} \approx 24 \text{ }\mu s$ before any software mitigation can respond.

Existing mitigations are not ideal and excessively sacrifice power/performance. Static voltage guard bands derate V_{nom} to accommodate worst-case droops, wasting power proportional to V^2 . Hardware power limiters (TDP throttles, clock gating) may be too coarse and apply too restrictive of a penalty on the entire workload, reducing the operating frequency well below achievable F_{max} . Neither approach exploits the fact that application-level batch size directly and continuously modulates GPU power draw.

SlewGuard is motivated by a simple observation: if software can shrink batch size M before a droop event deepens, V_{load} need never cross V_{min} . ResNet-18 inference is an ideal target because power scales linearly with M (each additional image adds $\sim 9 \text{ W}$), the workload is stateless across batches, and NVML provides real-time power telemetry with sufficient resolution to drive a PDN model. By closing the loop at the application layer (rather than the driver or hardware layer) SlewGuard achieves fine-grained, predictive droop suppression with no silicon modifications.

Proposed Methodology

SlewGuard implements a closed-loop feedback system with four components: a GPU workload backend, NVML-based power telemetry, a real-time RLC PDN model, and a hysteresis batch-size controller. The full data path is: NVML power $\rightarrow$ PDN model $\rightarrow$ V_{load} estimate $\rightarrow$ controller decision $\rightarrow$ batch size $M \rightarrow$ GPU workload.

Auto-Calibration: Before closed-loop operation, SlewGuard characterizes the workload by running it at M_{min} and M_{max} for 0.5 s each under NVML telemetry. From the measured P_{idle} (power at M_{min}), P_{full} (power at M_{max}), and power jitter σ , the controller derives:

- $V_{safe} = V_{min} + \text{headroom}$ (incorporates σ to guard against measurement noise);
- $V_{shrink} = V_{safe} - \text{deadband}$ (the single operational threshold, default deadband = 15 mV);
- $grow_step$: largest M increment whose resulting V_{ac} peak stays below V_{safe} , preventing a grow event from immediately triggering a shrink;
- $shrink_step$: fixed M decrement (default equal to $grow_step$) to rapidly reduce power when a droop is detected.

This auto-sizing eliminates manual per-workload tuning and generalizes across GEMM and ResNet-18 backends with different M ranges.

PDN Circuit Model: The PDN is modeled as a lumped RLC equivalent. Each 50 μs controller tick updates $V_{load}(t) = V_{nom} - V_{dc}(t) - V_{ac}(t)$, where $V_{dc} = I_{filt}(t) \cdot R$ is the DC resistive drop and V_{ac} captures the LC transient overshoot from the current step ΔI . Current I is derived from NVML power ($I = P / V_{nom}$). A first-order IIR envelope filter with time constant $\tau_{env} \approx 200 \mu\text{s}$ smooths NVML noise and models PDN settling between di/dt events. The PDN is pre-charged to its steady-state value at startup to eliminate inrush transients during the open-loop ramp phase.

Hysteresis Controller: The closed-loop phase runs on a 50 μs software tick (4 $\times$ faster than τ_{env}) ensuring the controller can observe and respond before PDN transients compound. Control follows a two-phase protocol:

- Phase 1 — Open-Loop Ramp: M increases linearly from M_{min} to M_{max} . The controller observes only. This warms up GPU caches, establishes thermal steady state, and pre-charges the PDN model.
- Phase 2 — Closed-Loop Control: Each tick, the controller evaluates the PDN model output.
 - If $V_{load} < V_{shrink}$ and $shrink_period$ has elapsed: $M \leftarrow M - shrink_step$. Reducing M lowers I and drives V_{dc} down, increasing V_{load} .
 - If $V_{load} \geq V_{shrink}$ and $grow_period$ (200 $\mu\text{s} = \tau_{env}$) has elapsed: $M \leftarrow M + grow_step$. Waiting one τ_{env} between grows ensures each V_{ac} undershoot decays before the next current step.
 - Otherwise: M is held constant.

The 15 mV deadband between V_{shrink} and V_{safe} prevents repetitive cycling around V_{shrink} . V_{load} must recover a full deadband margin before a grow is permitted after a shrink, suppressing constantly changing batch size near the threshold.

Workloads: Both kernels share a common Python interface ($run(M)$, $flops(M)$):

- GEMM Kernels (naive and tiled+GELU, $M \in [64, 512]$): compiled CUDA shared library with persistent GPU buffers (A , B , C pre-allocated at M_{max}). The tiled kernel maximizes compute intensity and has sharp di/dt events, representing a worst-case PDN stress scenario.
- ResNet-18 Inference ($M \in [1, 16]$): pretrained torchvision model in eval mode, input $[M, 3, 224, 224]$, 1.82 GFLOP/image (MACs $\times 2$ convention). A max batch tensor is pre-allocated once and sliced to $[:M]$ each run to remove cudaMalloc overhead for each call. CUDA events time each forward pass.

Results and Discussion

Effectiveness of batch throttling:

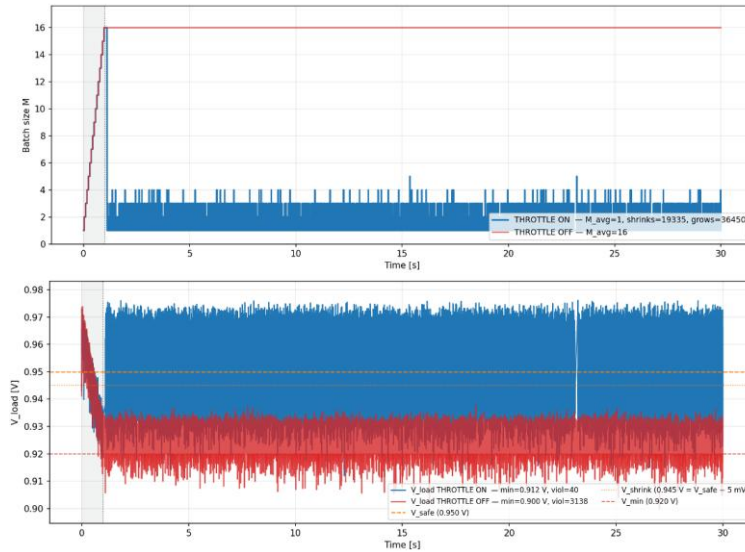


Figure 1: ResNet-18 Performance with dynamic throttling ON (blue) and OFF (red)

Without batch throttling (Throttle OFF), the workload experiences 3,138 $V_{\min}$ violations, leading to immediate system failure. Activating throttling reduces these violations to just 40, a 98.7% reduction. While throttling introduces batch size variability (averaging a batch size $M = 1$ -4 after the initial ramp), it enables a sustained throughput of 0.887 TFLOP/s. This confirms that throttling transforms a fatal power delivery issue into a manageable performance tradeoff.

Capacitive Sensitivity Analysis:

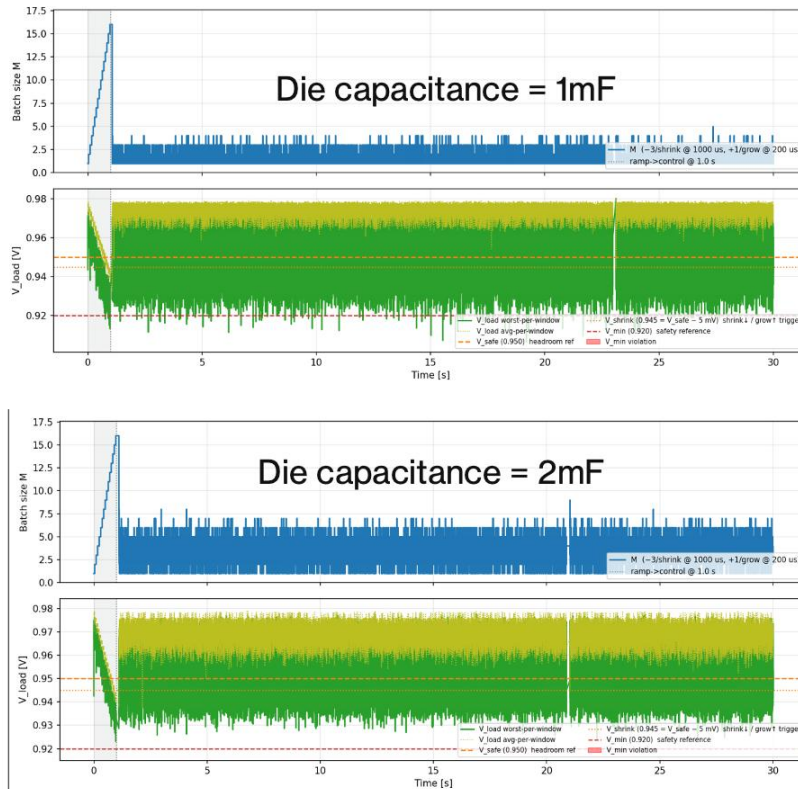


Figure 2: Batch Throttling with Die capacitance sweep

Capacitance acts as a primary buffer against high-frequency voltage droops. The sensitivity analysis reveals a non-linear benefit to increasing die capacitance:

- **Performance Scaling:** Doubling the die capacitance from 1mF to 2mF results in a massive throughput increase, jumping from 0.919 TFLOP/s to 2.022 TFLOP/s (a ~120% improvement).
- **Violation Elimination:** The higher capacitance provides sufficient local charge reservoirs to eliminate $V_{\min}$ violations entirely (reducing them from 110 to 0).
- **Throttling Behavior:** With 2mF, the system experiences significantly less frequent "shrinks" in batch size, allowing the hardware to maintain higher occupancy for longer durations.

Inductive Sensitivity Analysis:

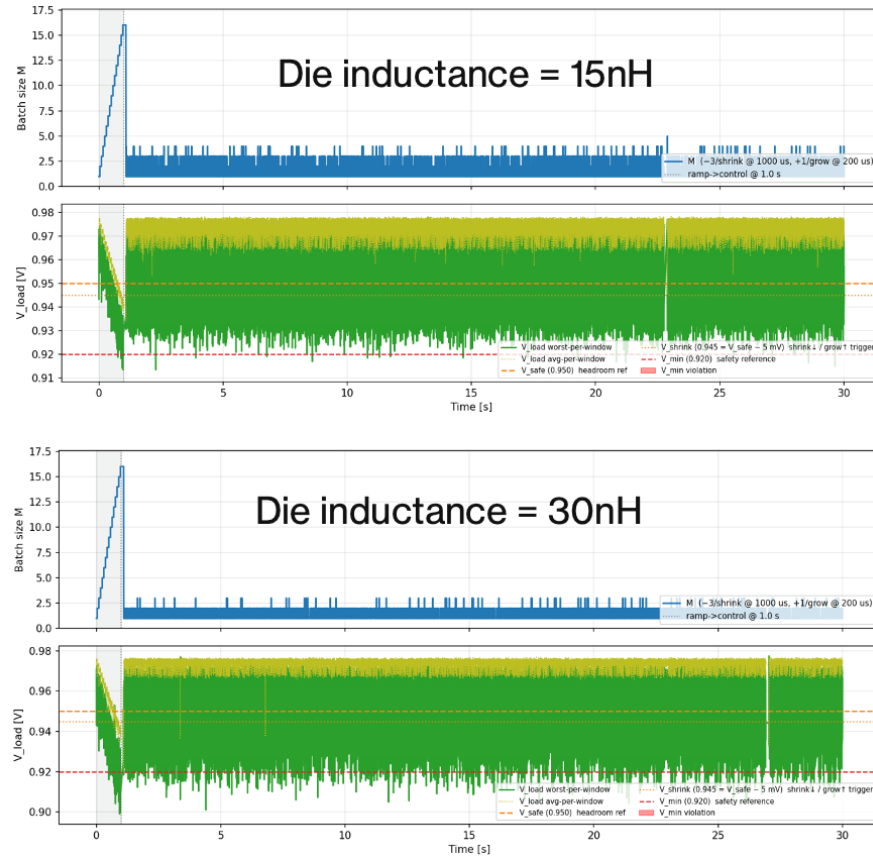


Figure 2: Batch Throttling with Die inductance sweep

Inductance is the primary antagonist to stable power delivery during rapid current changes (di/dt events) caused by neural network execution.

- **Increased Noise:** Increasing die inductance from 15nH to 30nH exacerbates high-frequency noise. This resistance to rapid current changes forces the voltage to drop more sharply during execution bursts.
- **Reliability Degradation:** The doubling of inductance causes $V_{\min}$ violations to skyrocket from 18 to 565, even with throttling active.
- **Performance Penalty:** The increased noise triggers more aggressive batch throttling to protect the system, resulting in a throughput drop from 0.859 TFLOP/s to 0.756 TFLOP/s.

Challenges and Learning Outcomes

Challenge 1: Controller grow-steps stacking and causing self-inflicted shrinks

Successive grow events compounded their V_{ac} undershoots, triggering the shrink path before the batch size (M) rises high enough. This prevented batch size from climbing higher above the minimum batch size ($M_{\min}$).

Fix: Enforce `grow_period >= tau_env` so each undershoot decays to near zero before the next grow fires

Learning: The timing between controller events matters as much as the thresholds themselves. A *fast* controller is not always a *stable* one.

Challenge 2: Narrow Batch Size Dynamic Range for ResNet-18

ResNet-18 operates over $M \in [1, 16]$ compared to the GEMM kernel's $M \in [64, 512]$ (8× range). Each unit increment of M corresponds to roughly 9 W of additional GPU power. Due to this large power increase auto-calibration must therefore set `grow_step = 1` (minimum) and `shrink_step = 1`, meaning the controller oscillates at the integer boundary of M and frequently saturates at $M_{min} = 1$. Under the default PDN parameters ($R = 0.3\text{ m}\Omega$, $L = 15\text{ nH}$, $C = 1\text{ mF}$), the throttled workload averages $M = 1\text{--}4$ and sustains only 0.887 TFLOP/s, a significant throughput penalty that would require better PDN hardware (larger C , lower L) to alleviate rather than software tuning alone.

Fix: Tune `grow_period` and `shrink_period` to the ResNet-18 kernel time (~2–5 ms at $M = 1\text{--}4$) rather than defaulting to `tenv = 200 μs`, slowing the controller to match actuator granularity.

Learning: PDN protection is fundamentally more expensive for small-batch CNN inference than for large-batch GEMM. A wider M range or better PDN hardware is required to recover throughput without sacrificing voltage safety.

Challenge 3: NVML Measurement Bandwidth vs. Controller Tick Rate

NVML reports GPU power at approximately 10 ms intervals (100 Hz), while the SlewGuard controller ticks every 50 μs (20 kHz), a 200× measurement bandwidth gap. For ResNet-18 at $M = 1\text{--}4$, a complete forward pass takes 2–5 ms, meaning multiple kernel launches complete between successive NVML power samples. The PDN model is therefore driven by stale power readings for the majority of its ticks, and rapid within-batch di/dt events (such as the power spike at the start of each convolutional layer) are invisible to the controller until the next NVML sample arrives.

Fix: The `--pdn-noise-scale` parameter injects Gaussian noise proportional to the measured NVML jitter σ into each tick's power estimate, statistically modeling unobserved within-sample variation. For deterministic reproducibility, `--pdn-noise-scale 0` disables injection.

Learning: A production deployment would require a faster power telemetry channel (e.g., on-die power sensors) to close the measurement latency gap. NVML is a best-effort approximation suitable for proof-of-concept validation but insufficient for tight voltage margin control at sub-millisecond granularity.

Conclusion and Future Work

While dynamic batch throttling is an effective insurance policy that prevents system crashes, it is not a substitute for high-quality hardware design. Relying solely on throttling leads to significant performance degradation (lower TFLOP/s). The data proves that power integrity is a dual-domain challenge requiring both firmware and hardware solutions. The interaction between PDN parasitics and throttling logic leads to several key conclusions for system design:

Parameter	Change	Impact on Vmin Violations	Performance Impact
Batch Throttling	OFF → ON	Massive Reduction (~3000 to 40)	Enables functional execution
Capacitance	1mF → 2mF	Eliminated (110 to 0)	+120% TFLOP/s
Inductance	15nH → 30nH	Massive Increase (18 to 565)	-12% TFLOP/s

Table 1: Summary of Findings

While dynamic throttling can protect the system against larger sustained voltage droops, the PDN is the only line of defense against high-frequency, nanosecond-scale noise events that occur too fast for software to detect or mitigate. True performance maximization is achieved by optimizing the PDN, specifically by increasing die capacitance and reducing inductance to minimize the need for throttling, thereby allowing the chip to operate at its highest possible throughput (TFLOPS) with a safe, narrow voltage guard band.

Future work worth exploring:

Predictive model controller: Replacing the existing 2-state hysteresis controller with a lightweight predictive controller that can forecast the Vload over the next N ticks and pre-emptively issue shrinks before crossing Vshrink will help in faster batch size recovery and reduce the performance (TFLOPS) penalty.

Team Contributions

1. Prototyping batch-size driven power sensitivity using PyNVML polling to lay the groundwork for Slew Guard: **Jakub**
2. Power Delivery Network (PDN) and Controller System Design: **Nikhil**
3. Controller Validation with worst-case tiled GEMM kernels to establish impact of dynamic throttling: **Nikhil**
4. ResNet-18 workload design and integration into Slew Guard flow: **Jakub**
5. Optimize the PDN to characterize RLC effects and their interaction with dynamic throttling on ResNet-18 performance: **Nikhil and Jakub**
6. Compile performance results and reports: **Nikhil and Jakub**

References

[1] Z. Liang et al., “GPU-To-Grid: Understanding and Mitigating the Impact of GPU Workloads on Power Delivery Networks,” *arXiv:2602.05116*, 2025.